

MEMÒRIA PRÀCTICA 3

Assignatura: Programació 2

Curs: 2025-2026

Autors: [GarciaMarcos] i [ChoudnaEmad]

1. Implementació d'una altra classe Dades, d'un altre company.

- **quines modificacions serien necessàries en la vista i l'adaptador?**

Cap. La vista (per exemple AppBiblioUB.java) no sap res de la classe Dades ni de com es guarda la informació. Només parla amb l'Adaptador, cridant mètodes com adaptador.guardaDades(rutaFitxer). Per tant, el que faci el meu company per dins no li afecta gens.

Poques o cap. A Adaptador.java tinc:

```
private Dades dades;
```

```
this.dades = new Dades();
```

Si el meu company ha anomenat la classe igual (Dades.java), simplement substitueixo el fitxer i ja funciona. Si li ha posat un altre nom, canvio aquella línia i prou.

Una cosa que milloraria el disseny seria declarar l'atribut com private InDades dades; en lloc d'usar la classe concreta, així l'adaptador dependria de la interfície i no de la implementació concreta.

- **Com es relaciona la interfície InDades amb la pregunta anterior?**

La interfície és el que fa que intercanviar el codi sigui segur. Obliga a qualsevol classe que la implementi a tenir els mateixos mètodes, amb els mateixos paràmetres i tipus de retorn. Així, si l'adaptador crida dades.recuperaUsuaris(), sabem segur que aquest mètode existirà a la classe del meu company i que retornarà un ArrayList<Usuari>.

2. Quines conseqüències tindria que el mètode recuperaExemplars de la classe Adaptador retornés una llista d'objectes de la classe Exemplar en lloc d'una llista de String?

Si el mètode recuperaExemplars retornés directament una llista d'objectes Exemplar, la Vista hauria d'importar aquesta classe i cridar mètodes com

getTitol() o getAutor() per poder mostrar res per pantalla. Això és un problema perquè la Vista deixaria de ser independent de la lògica interna.

L'Adaptador existeix precisament per evitar això: s'encarrega de convertir les dades en Strings simples perquè la Vista no hagi de saber res de com funciona el programa per dins. Si la Vista coneix la classe Exemplar, qualsevol canvi en aquesta classe (per exemple, canviar el nom d'un atribut) obligaria a tocar també el codi de la interfície gràfica, quan no hauria de tenir res a veure.

3. Expliqueu quines classes has pogut reutilitzar de la pràctica 3 part 1 per a fer la part 2. Quins canvis sobre les classes reutilitzades has necessitat fer i perquè.

Per a la segona part de la pràctica hem pogut reutilitzar gairebé tot el que teníem. El paquet model l'hem conservat íntegrament: Dades, Exemplar, Usuari, Prestec, Llista i totes les seves classes filles segueixen igual. De la vista, l'única cosa que hem mantingut és BiblioException, ja que la gestió d'excepcions no canvia. En canvi, IniciadorBiblioUb, BiblioUB i Menu les hem eliminat perquè tota la interfície de terminal ja no té sentit amb l'entorn gràfic de Swing.

El canvi principal l'hem fet a Adaptador. Els mètodes que retornaven informació per mostrar llistes, com mostrarExemplars, mostrarUsuaris i mostrarPrestecs, ara iteren sobre les dades i retornen un ArrayList<String> usant el .toString() de cada objecte. Això ho hem fet perquè les finestres gràfiques només rebin text simple i no hagin d'importar ni conèixer les classes del model.

4. Indiqueu quins tipus d'esdeveniments heu fet servir al vostre codi.

Hem fet servir principalment tres tipus d'esdeveniments. El més utilitzat és l'ActionEvent amb ActionListener, que es dispara quan l'usuari clica un botó (JButton), ja sigui per obrir un formulari, acceptar, cancel·lar o tornar enrere. També l'hem usat per detectar canvis a les caselles de verificació (JCheckBox), concretament per filtrar la llista de préstecs no retornats.

Després hem usat el WindowEvent amb WindowAdapter per gestionar quan l'usuari tanca una subfinestra amb la creueta (windowClosing), fent que s'executi la mateixa lògica que si hagués premut cancel·lar.

Finalment, per tancar els formularis amb la tecla Escape hem usat `registerKeyboardAction`, que ens ha semblat la manera més còmoda de fer-ho sense complicar el codi.

5. Explica quins canvis hauríeu de realitzar en l'aplicació si es volgués afegir la funcionalitat següent: En iniciar l'aplicació calobrir-ne una subfinestra amb la llista de préstecs pendents d'enviar.

Per fer això caldria tocar les tres capes. Al Model, a la classe `Dades`, afegiríem un mètode tipus `recuperaPrestecsPendentsEnviar()` que recorregués la llista de préstecs i retornés només els que complissin el criteri. A l'Adaptador, faríem un mètode equivalent que cridés aquest i convertís els resultats a `ArrayList<String>` amb el `.toString()`, per no passar objectes del model directament a la vista.

A la Vista crearíem un nou `JDialog`, per exemple `FrmPrestecsPendents`, amb un `JList` o `JTable` per mostrar les dades. I per fer que s'obri automàticament en arrencar l'aplicació, modificariem `AppBiblioUB`: al final del constructor o dins del `SwingUtilities.invokeLater`, instanciariem aquest diàleg, li passariem l'adaptador, cridaríem el mètode per carregar les dades i faríem `setVisible(true)`.

6. Proves realitzades per comprovar el correcte funcionament de la pràctica, resultats obtinguts i accions derivades.

Mòdul / Finestra	Què hem provat?	Què va passar?	Com ho hem solucionat?
Gestió d'Usuaris	Deixar els camps Nom o Email en blanc i intentar afegir l'usuari.	Es creava l'usuari igualment, amb dades buides.	Ara a <code>FrmAfegirUsuari</code> es comprova <code>nom.isEmpty()</code> abans de res i surt un <code>JOptionPane</code> d'avís.
Gestió de Préstecs	Intentar prestar un exemplar que ja el té un altre usuari.	<code>afegirPrestec</code> llençava una <code>BiblioException</code> i petava per consola.	Hem posat un <code>try-catch</code> a <code>FrmAfegirPrestec</code> i ara l'error es mostra amb un diàleg en lloc de petar.
Persistència	Carregar un <code>.dat</code> que no existeix o que està corrupte.	Saltava una <code>FileNotFoundException</code> o <code>ClassNotFoundException</code> sense controlar.	Hem capturat els errors a l'adaptador i la vista ara avisa l'usuari del que ha passat.
Interfície Gràfica	Maximitzar la finestra principal (<code>AppBiblioUB</code>).	Els botons s'estiraven horitzontalment i quedava tot deformat.	Hem agrupat els botons en un panell central amb <code>Spacers</code> i li hem posat un <code>Maximum Size fix</code> .

Mòdul / Finestra	Què hem provat?	Què va passar?	Com ho hem solucionat?
Gestió de Préstecs	Tancar la subfinestra d'Afegir Préstec amb la "X" en lloc del botó Cancel·lar.	Es tancava però sense passar pel dispose(), cosa que podia deixar coses penjades.	Hem afegit un WindowAdapter amb windowClosing perquè faci el mateix que el Cancel·lar.
Filtres (Préstecs)	Marcar i desmarcar el CheckBox de "Només préstecs no retornats".	La llista no s'actualitzava sola, calia prémer un botó a part per refrescar.	Hem posat un ActionListener directament al JCheckBox perquè cridi actualitzarLlista() sol.

7. Observacions generals.

En general, el canvi de terminal a interfície gràfica amb Swing ha estat el que més ens ha costat d'aquesta part. No tant per la dificultat tècnica en si, sinó per haver de pensar diferent: en un terminal el flux és lineal i controlat, però amb finestres l'usuari pot fer qualsevol cosa en qualsevol moment, i has de tenir-ho tot previst.

El que més hem après és que tenir el codi ben separat entre Vista, Adaptador i Model no és només teoria. Quan hem hagut de fer canvis o corregir errors, el fet que la Vista no sàpiga res del Model ha fet que els problemes quedessin aïllats i fossin molt més fàcils de localitzar.

L'altra cosa que ens ha ocupat més temps del previst ha estat la gestió d'errors. Deixar camps buits, tancar finestres amb la "X", intentar carregar fitxers que no existeixen... cadascun d'aquests casos ha necessitat la seva validació o el seu try-catch, i fins que no ho has provat tot de veritat no te n'adones de quantes coses poden sortir malament.